

CS6650 Assignment 4

Distributed Databases using Replication

Emily Chen | Spring 2025

Northeastern University – Khoury College of Computer Sciences

This report presents the design, implementation, and experimental evaluation of two distributed key-value store systems: a **Leader-Follower** database and a **Leaderless** database. Three replication strategies are evaluated for the Leader-Follower model ($W=5/R=1$, $W=1/R=5$, and $W=3/R=3$ quorum), and one for Leaderless ($W=N/R=1$). All four configurations are load-tested across four read/write ratios and consistency properties are verified through unit tests.

1. System Architecture and Code Explanation

The entire system compiles into a single Go binary (`cmd/node/main.go`) configured entirely by environment variables at startup. The same Docker image runs every node type – leader, follower, or leaderless – and Docker Compose passes different env vars to each container. The three key variables are `ROLE` (leader/follower/leaderless), `MODE` (`w5r1/w1r5/quorum`, leader only), and `PEERS` (comma-separated base URLs of peer nodes).

1.1 Core KV Store (`internal/store/store.go`)

Every node holds an in-memory map protected by a `sync.RWMutex`. Each entry contains a `Value` string and a `Version` `int64` – a logical clock starting at 1 on the first write and incrementing by 1 on every subsequent write. When a follower receives a replication message it uses the version assigned by the leader rather than computing its own, ensuring all nodes agree on version numbers. The mutex uses `RLock` for concurrent reads and `exclusive Lock` for writes.

1.2 Leader-Follower (`internal/leader/handler.go`)

The `LeaderHandler` controls all client-facing write and read logic. The `FollowerHandler` only exposes internal replication endpoints – client writes to a follower are rejected with `HTTP 403`, enforcing that all writes flow through the leader.

What happens on a write (set)?

Mode	What the leader does on set	Client gets 201 when...
$W=5\ R=1$	Sends to all 4 followers, waits for all 4 acks	All 5 nodes written (~810ms)
$W=1\ R=5$	Writes locally only. Replication fires in background goroutine.	Leader local write completes (~1ms)
$W=3\ R=3$	Waits for 2 followers. Remaining 2 updated asynchronously.	3 nodes written (~405ms)

What happens on a read (get)?

For $W=5/R=1$ the leader returns its own local value immediately ($R=1$). Since all writes

committed to all nodes before returning, the local value is always fresh. For $W=1/R=5$ the leader fans out HTTP GETs to all 4 followers concurrently, collects all responses, and returns the entry with the highest version number – this corrects stale follower values at read time. For $W=3/R=3$ the leader fetches from itself plus 2 followers (3 total). Because $W+R=6 > N=5$, the write and read quorums always overlap by at least one node, guaranteeing the most recent version is always in the read set.

The 200ms sleep after each follower message and 100ms sleep inside each follower's replicate handler are deliberate delays required by the assignment. They widen the inconsistency window so stale reads are observable during testing.

1.3 Leaderless ([internal/leaderless/handler.go](#))

Every node is identical. When a node receives a /set request it becomes the Write Coordinator. It writes locally first (establishing the canonical version), then fans out replication to all 4 peers using goroutines, sleeping 200ms between each dispatch, and waits for all 4 to acknowledge ($W=N=5$) before returning 201. Reads are served purely locally ($R=1$) with no coordination – if the replication has not yet reached a node, reads return stale data.

1.4 Error Handling

All peer HTTP calls run in goroutines with error logging. If a follower is unreachable the error is logged and that goroutine exits without blocking others. The load tester uses a 10-second HTTP client timeout to prevent hangs under failure conditions.

1.5 Load Tester Design ([loadtest/main.go](#))

The load tester uses a small fixed key pool (20 keys) shared across all concurrent workers. With 10 workers hitting 20 keys, the same keys receive reads and writes within milliseconds of each other, maximising stale read exposure. Two thread-safe maps track state: a version tracker (key → highest written version) for staleness detection, and a write-time tracker (key → unix-ms of last write) for computing read-write intervals.

For leader-follower configurations, all reads go to the leader. The leader then handles R internally – $R=1$ returns local data, $R=5$ fans out to all 4 followers, $R=3$ fans out to 2 followers. Sending reads directly to followers would bypass the quorum logic entirely. For leaderless, reads go to any random node ($R=1$ local read per the assignment spec).

2. Load Test Results

2.1 Latency CDFs – All Configurations

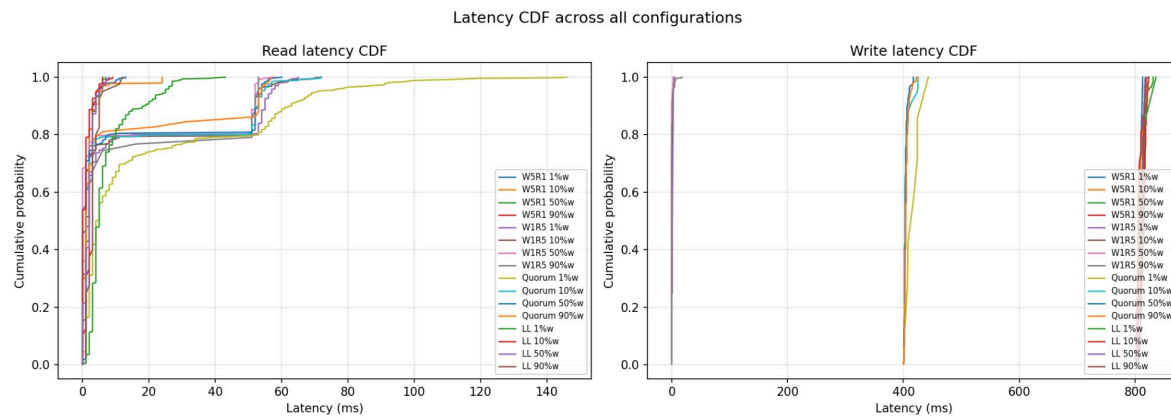


Figure 1. Read latency CDF (left) and write latency CDF (right) across all 16 configurations. Two distinct read clusters are visible: W5R1/Leaderless under 10ms, W1R5/Quorum around 50ms.

2.2 Per-Ratio Results – 1% Writes

Latency Results - 1% writes

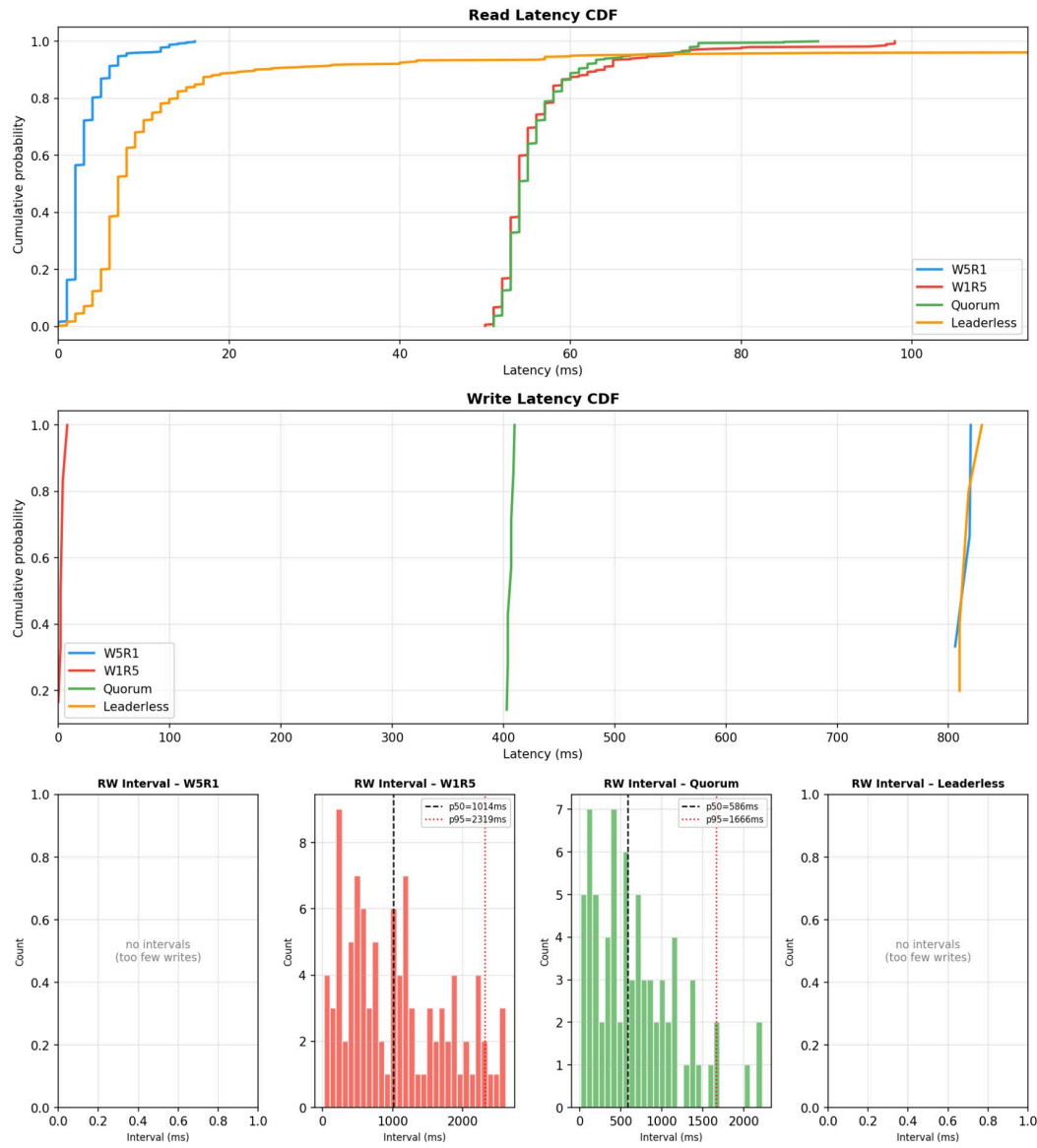


Figure 2. Read CDF, Write CDF, and RW interval distribution at 1% writes. W5R1 and Leaderless dominate with near-instant reads. W1R5 has no visible write bar (sub-1ms).

2.3 Per-Ratio Results – 10% Writes

Latency Results - 10% writes

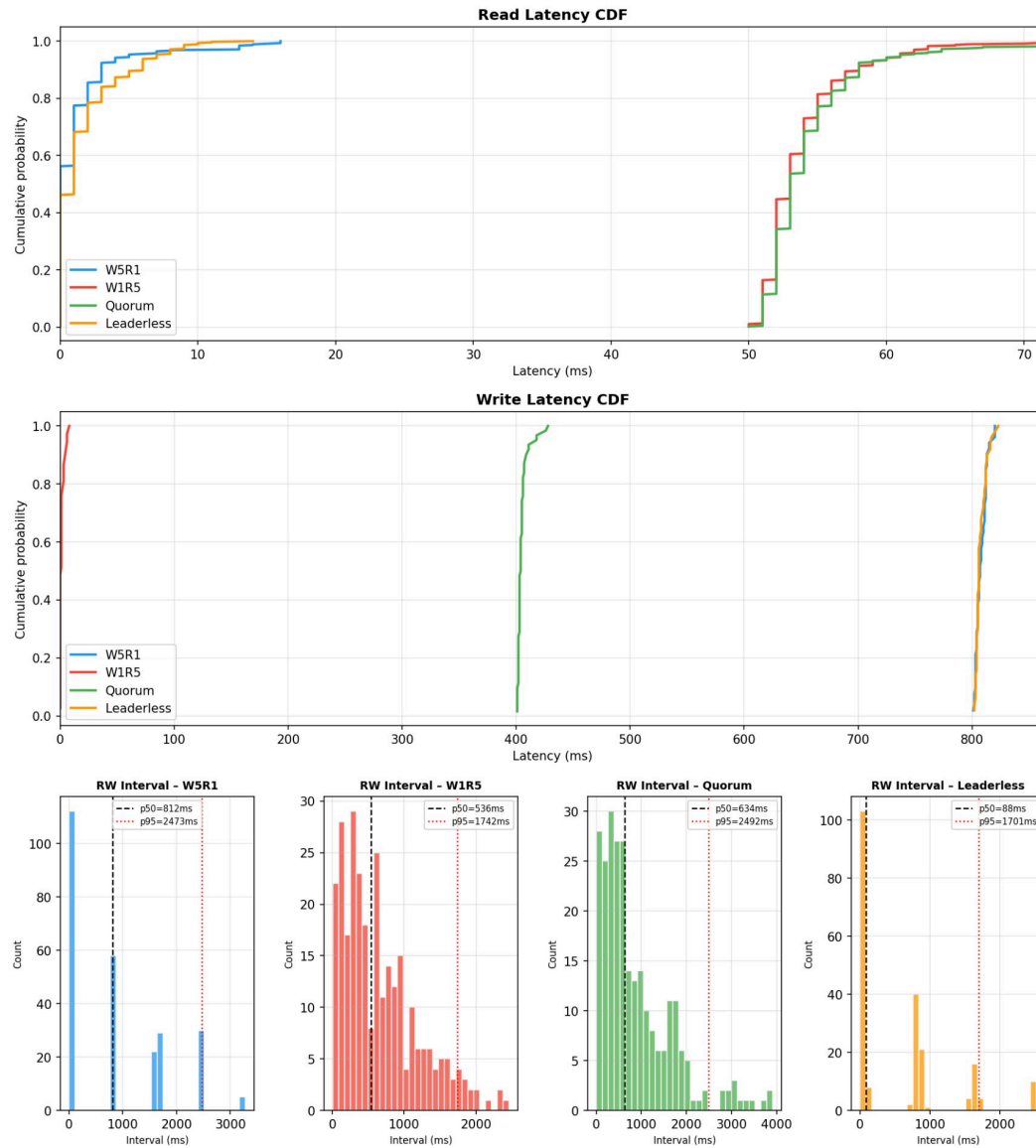


Figure 3. At 10% writes, the two read latency clusters separate clearly. W1R5 and Quorum reads cluster at ~52ms; W5R1 and Leaderless under 5ms.

2.4 Per-Ratio Results – 50% Writes

Latency Results - 50% writes

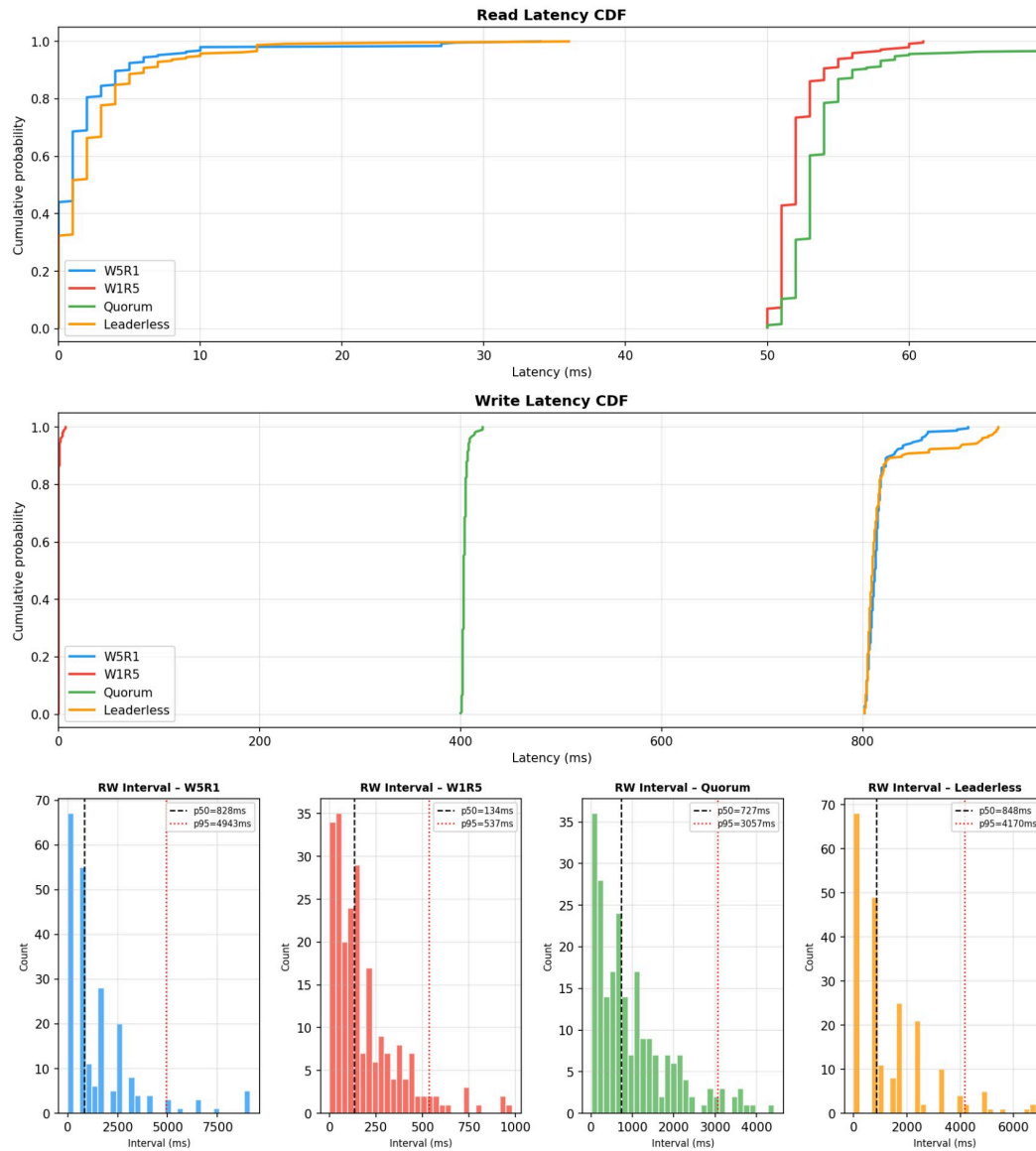


Figure 4. At 50% writes, the write latency long tail for W5R1/Leaderless becomes visible (p95 approaching 900ms). W1R5 write CDF is still near-vertical at 0ms.

2.5 Per-Ratio Results – 90% Writes

Latency Results - 90% writes

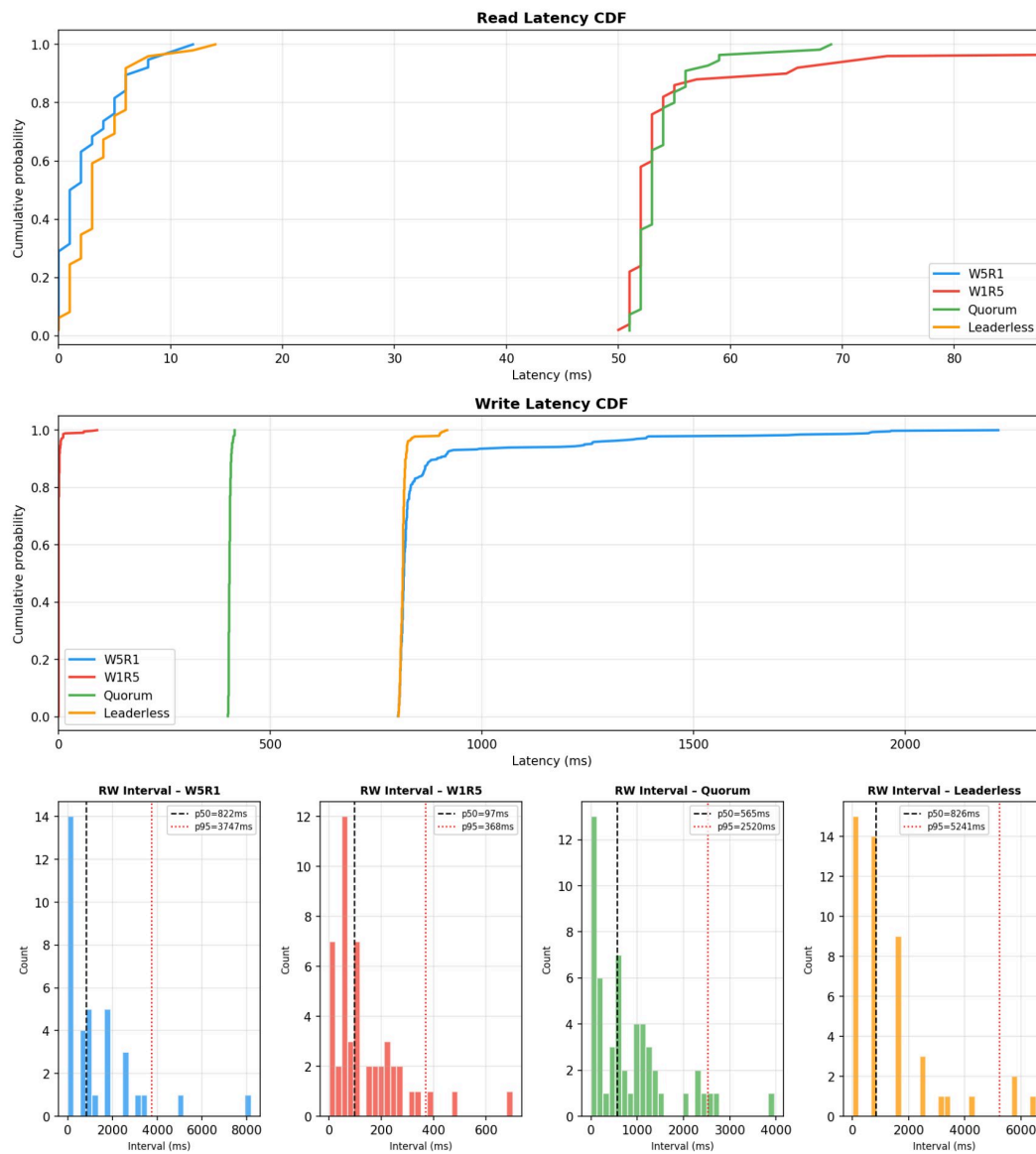


Figure 5. At 90% writes, W5R1 shows significant write latency spread (800–2200ms). This is the long tail – concurrent writes compete for the same follower connections, increasing variance.

2.6 Average Read Latency

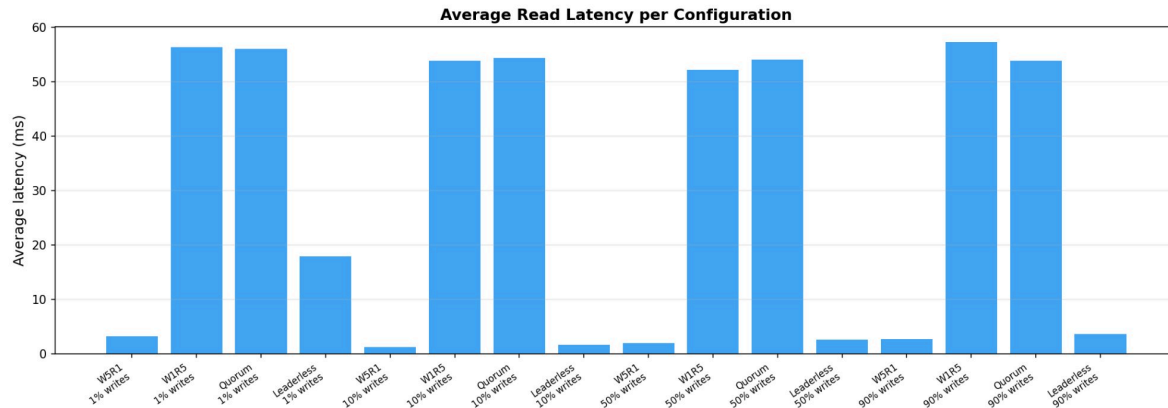


Figure 6. Average read latency per configuration. W5R1 and Leaderless consistently under 5ms ($R=1$ local reads). W1R5 and Quorum consistently around 54ms (fan-out with 50ms follower sleep).

2.7 Average Write Latency

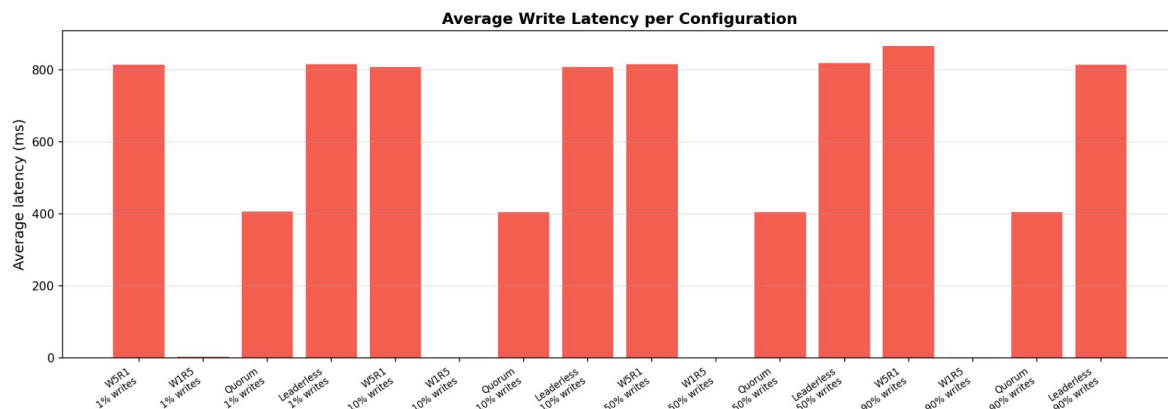


Figure 7. Average write latency per configuration. W1R5 is invisible (sub-1ms). Quorum clusters at ~405ms. W5R1 and Leaderless cluster at ~815ms.

2.8 Stale Read Rate

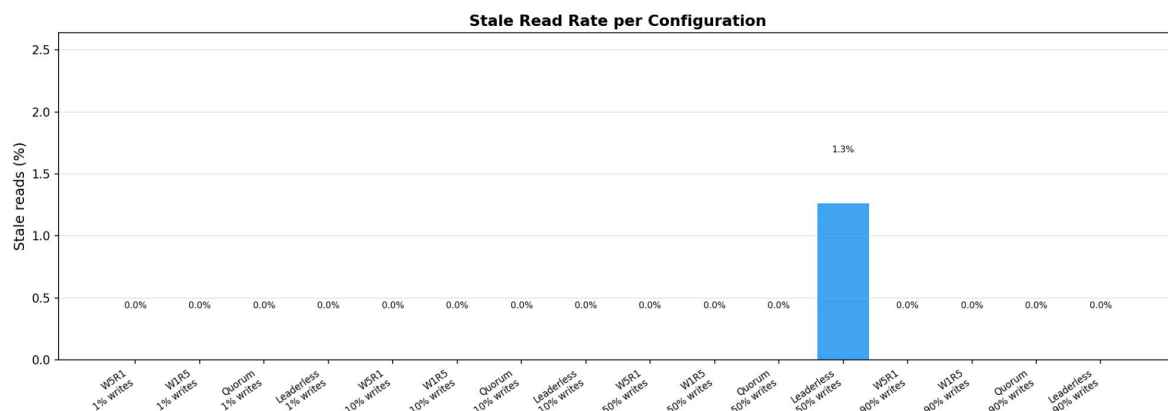


Figure 8. Stale read rate per configuration. Only Leaderless at 50% writes shows any staleness (1.3%). All other configurations show 0% stale reads.

3. Discussion

3.1 Write Latency

W=5/R=1 and Leaderless both cluster at ~810ms (Figures 6–7). This is the minimum imposed by the artificial delays: 4 followers × 200ms leader sleep + 100ms follower write = 900ms theoretical minimum, with network overhead reducing it slightly to ~810ms average. Every write takes almost exactly the same time because all four peer confirmations must arrive before returning 201. At 90% writes (Figure 5), the write CDF spreads between 800–2200ms, showing a long tail caused by concurrent writes competing for follower connections.

Quorum (W=3/R=3) writes cluster at ~405ms – exactly half the W=5 cost because the leader waits for only 2 followers (2 × 200ms) before responding. The remaining 2 followers are updated asynchronously. This is the direct payoff of reducing W: halving the synchronous replication factor halves write latency.

W=1/R=5 writes are sub-1ms across all ratios – the leader writes locally and returns immediately. The W1R5 bar is invisible in Figure 7 because it is too small to render at the 0–900ms scale.

3.2 Read Latency

W=5/R=1 and Leaderless reads complete in under 5ms across all ratios (Figures 2–6). Since R=1, reads are local map lookups with no network overhead. The CDF rises almost vertically near zero in every per-ratio figure.

W=1/R=5 and Quorum reads cluster at ~52–58ms. The bottleneck is the 50ms follower sleep on internal reads, which runs in parallel across all polled nodes – so the read latency equals the slowest responding node plus network overhead. R=5 (4 followers) and R=3 (2 followers) have nearly identical read latency because the slowest follower in both cases sleeps the same 50ms.

3.3 Stale Reads

Figure 8 shows near-zero stale reads across almost all configurations. This is the correct behaviour for each mode:

- **W=5/R=1:** All 5 nodes are written before 201 is returned, so any subsequent read to the leader finds the latest version immediately.
- **W=1/R=5:** The leader fans out to all 5 nodes on read and returns the highest version. Even if followers are behind, the leader always picks the freshest copy – so the client never observes stale data despite writes being asynchronous.
- **Quorum W=3/R=3:** $W+R=6 > N=5$ guarantees the read set always overlaps the write set by at least one node, so the highest version is always present in the R=3 responses.
- **Leaderless W=N/R=1:** All 5 nodes are updated before 201 is returned. The 1.3% stale reads at 50% writes occur from concurrent writes to the same key where one write completes and a read arrives at a node before the second concurrent write has propagated.

The 1.3% leaderless stale rate at 50% writes is a genuine race condition: two workers writing the same key simultaneously. Worker A finishes and updates the version tracker. Worker B's write is still mid-propagation. A read arrives and finds B's old version on one node. This is expected behaviour for a leaderless system without locking across coordinators.

3.4 Read-Write Interval Distribution

The interval histograms (bottom row of Figures 2–5) show how quickly reads follow writes on the same key. This is the mechanism that produces stale reads.

W=1/R=5 at 10% writes (Figure 3): p50 interval = 536ms, well within the 800ms replication window. However since the leader always returns the highest version from R=5 nodes, these reads are not stale despite landing close to the write. The interval histogram explains why W=1/R=5 requires careful implementation – if reads went to followers directly instead of the leader, the stale rate would be extremely high.

W=5/R=1 at 10% writes (Figure 3): p50 interval = 812ms. Reads almost always follow writes by more than the replication window because writes take ~810ms themselves. By the time a write completes, all nodes are already updated.

Leaderless at 10% writes (Figure 3): p50 interval = 88ms, much shorter than the 810ms propagation window. This explains the intermittent stale reads – reads arrive before propagation completes on some nodes.

3.5 Which Leader-Follower Mode Does Best at Each Ratio?

Write %	Best Overall	Worst	Reasoning
1% writes	W=5/R=1	W=1/R=5	Almost all traffic is reads. W=5/R=1 delivers ~2ms reads with zero staleness. W=1/R=5 reads cost 54ms with no benefit at low write rates.
10% writes	W=5/R=1	W=1/R=5	Still read-dominant. W=5/R=1 fast reads dominate. The ~810ms write cost appears only 10% of the time. W=1/R=5 read fan-out cost is unjustified.
50% writes	Quorum W=3/ R=3	W=5/R=1	Balanced load. W=5 write cost (~810ms) becomes significant at this ratio. Quorum halves write latency (~405ms) while reads stay consistent. W=5/R=1 throughput suffers under write load.
90% writes	W=1/R=5	W=5/R=1	Write-dominated. W=1/R=5 writes at sub-1ms maximise throughput. The few reads still return the freshest version via R=5 fan-out. W=5/R=1 at 810ms/write would saturate the system.

3.6 Which Database for Which Application?

Mode	Best Application	Why
LF W=5/R=1	Read-heavy, strong consistency. Product catalog, user profiles, DNS.	Reads are ~2ms and always consistent (R=1 from always-updated leader). Zero stale reads. Write cost (~810ms) is acceptable when writes are rare.
LF W=1/R=5	Write-heavy with freshness guarantee. Real-time dashboards, financial data feeds.	Writes are sub-1ms. Reads always return the most recent version by scanning all nodes. High read latency (~54ms) acceptable for dashboards. Zero stale reads despite async replication.

LF $W=3/R=3$ Quorum	Mixed read/write workloads. Shopping cart, session state, collaborative editing.	Best balance: write latency halved vs $W=5$, reads consistent via quorum overlap guarantee ($W+R>N$). General-purpose choice for most applications.
Leaderless $W=N/R=1$	Multi-region or high-availability writes. IoT data collection, edge computing.	No single point of failure – any node can accept writes. Reads are local and fast ($\sim 2\text{ms}$). Suitable when write availability matters more than write latency.